

Санкт-Петербургский Национальный Исследовательский Университет
Информационных Технологий, Механики и Оптики

Факультет Инфокоммуникационных Технологий

Web-программирование

Лабораторная работа 6

Выполнил

Панаёт В. Т.

Проверила

Марченко Е. В.

Санкт-Петербург

2025

Цель: освоить основы работы с сетевыми сокетами и многопоточностью в Python через разработку различных клиент-серверных приложений.

Задачи:

- Реализация простейшего клиент-серверного приложения
- Реализация клиент-серверного приложения для выполнения математической операции
- Реализация серверной части HTTP-приложения
- Реализация многопользовательского чата

Задание 1

В данном задании реализована базовая клиент-серверная модель:

- Клиент отправляет серверу сообщение «Hello, server».
- Сервер получает сообщение, выводит его и отправляет обратно клиенту сообщение «Hello, client».
- Клиент выводит полученное сообщение.

Код сервера для получения сообщения и отправки ответа. представлен на рисунке 1:

```
import threading
import logging

HOST = '127.0.0.1'
PORT = 65432

def handle_client(conn, addr): 1 usage
    logging.info(f"Подключен клиент: {addr}")
    try:
        data = conn.recv(1024)
        if data:
            message = data.decode('utf-8')
            logging.info(f"Получено от клиента {addr}: {message}")
            if message == "Hello, server":
                response = "Hello, client"
                conn.sendall(response.encode('utf-8'))
    except Exception as e:
        logging.error(f"Ошибка при работе с клиентом {addr}: {e}")
    finally:
        conn.close()

def main(): 1 usage
    logging.basicConfig(level=logging.INFO)
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
        server_socket.bind((HOST, PORT))
        server_socket.listen()
        logging.info(f"Сервер запущен и прослушивает {HOST}:{PORT}")
        try:
            while True:
                conn, addr = server_socket.accept()
                client_thread = threading.Thread(target=handle_client, args=(conn, addr))
                client_thread.daemon = True
                client_thread.start()
        except KeyboardInterrupt:
            logging.info("Сервер остановлен вручную.")
```

Рисунок 1 – Код server.py

Код клиента, отправляющего сообщение и получающего ответ, представлен на рисунке 2:

```
import socket
import logging

HOST = '127.0.0.1'
PORT = 65432

def main(): 1 usage
    logging.basicConfig(level=logging.INFO)
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client_socket:
            client_socket.connect((HOST, PORT))
            message = "Hello, server"
            client_socket.sendall(message.encode('utf-8'))
            data = client_socket.recv(1024)
            logging.info(f"Получено от сервера: {data.decode('utf-8')}")
    except Exception as e:
        logging.error(f"Ошибка соединения с сервером: {e}")

if __name__ == '__main__':
    main()
```

Рисунок 2 – Код client.py

Пример использования, отправка сообщения и получение клиентом представлен на рисунках 3 и 4:

```
INFO:root:Получено от сервера: Hello, client
```

Рисунок 3 – Результат работы клиента

```
INFO:root:Сервер запущен и прослушивает 127.0.0.1:65432
INFO:root:Подключен клиент: ('127.0.0.1', 62737)
INFO:root:Получено от клиента ('127.0.0.1', 62737): Hello, server
```

Рисунок 4 – Результат работы сервера

Задание 2

Реализована система, где клиент запрашивает выполнение математической операции. Возможны следующие варианты:

- Теорема Пифагора;
- Решение квадратного уравнения;
- Поиск площади трапеции;
- Поиск площади параллелограмма.

Клиент вводит с клавиатуры необходимые параметры, отправляет их серверу, который обрабатывает данные и возвращает результат.

Был написан сервер, который принимает операцию и параметры для расчета математических операций. На рисунках 5, 6, 7, 8, 9 представлен код сервера.

```
if operation == "trapezoid":
    if len(parts) != 4:
        response = "Ошибка: для трапеции нужны 3 параметра (база1, база2, высота)"
    else:
        base1, base2, height = map(float, parts[1:4])
        area = ((base1 + base2) * height) / 2
        response = f"Площадь трапеции: {area}"
```

Рисунок 5 – Расчет площади трапеции

```
elif operation == "parallelogram":
    if len(parts) != 3:
        response = "Ошибка: для параллелограмма нужны 2 параметра (основание, высота)"
    else:
        base, height = map(float, parts[1:3])
        area = base * height
        response = f"Площадь параллелограмма: {area}"
```

Рисунок 6 – Расчет площади параллелограмма

```
elif operation == "pythagoras":
    if len(parts) != 3:
        response = "Ошибка: для теоремы Пифагора нужны 2 параметра (катеты)"
    else:
        a, b = map(float, parts[1:3])
        c = math.sqrt(a**2 + b**2)
        response = f"Гипотенуза: {c}"
```

Рисунок 7 – Расчет теоремы Пифагора

```
elif operation == "quadratic":
    if len(parts) != 4:
        response = "Ошибка: для квадратного уравнения нужны 3 параметра (a, b, c)"
    else:
        a, b, c = map(float, parts[1:4])
        discriminant = b**2 - 4 * a * c
        if discriminant < 0:
            response = "Нет вещественных корней"
        elif discriminant == 0:
            x = -b / (2 * a)
            response = f"Один корень: {x}"
        else:
            x1 = (-b + math.sqrt(discriminant)) / (2 * a)
            x2 = (-b - math.sqrt(discriminant)) / (2 * a)
            response = f"Два корня: {x1} и {x2}"
else:
    response = "Ошибка: неизвестная операция"
```

Рисунок 8 – Решение квадратного уравнения

```
def main(): 1 usage
    logging.basicConfig(level=logging.INFO)
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
        server_socket.bind((HOST, PORT))
        server_socket.listen()
        logging.info(f"Сервер запущен и слушает {HOST}:{PORT}")
        try:
            while True:
                conn, addr = server_socket.accept()
                client_thread = threading.Thread(target=handle_client, args=(conn, addr))
                client_thread.daemon = True
                client_thread.start()
        except KeyboardInterrupt:
            logging.info("Сервер остановлен вручную.")
```

Рисунок 9 – Реализация сервера

На 10 и 11 рисунке представлена реализация клиента:

```
def main(): 1 usage
    print("Выберите операцию:")
    print("1. Площадь трапеции")
    print("2. Площадь параллелограмма")
    print("3. Теорема Пифагора")
    print("4. Решение квадратного уравнения")
    choice = input("Введите номер операции: ")

    if choice == '1':
        operation = "trapezoid"
        base1 = input("Введите первую базу трапеции: ")
        base2 = input("Введите вторую базу трапеции: ")
        height = input("Введите высоту трапеции: ")
        message = f"{operation},{base1},{base2},{height}"
    elif choice == '2':
        operation = "parallelogram"
        base = input("Введите основание параллелограмма: ")
        height = input("Введите высоту параллелограмма: ")
        message = f"{operation},{base},{height}"
    elif choice == '3':
        operation = "pythagoras"
        a = input("Введите первый катет: ")
        b = input("Введите второй катет: ")
        message = f"{operation},{a},{b}"
    elif choice == '4':
        operation = "quadratic"
        a = input("Введите коэффициент a: ")
        b = input("Введите коэффициент b: ")
        c = input("Введите коэффициент c: ")
        message = f"{operation},{a},{b},{c}"
    else:
        print("Неверный выбор операции")
        return
```

Рисунок 10 – Считывание пользовательского ввода

```

try:
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client_socket:
        client_socket.connect((HOST, PORT))
        client_socket.sendall(message.encode('utf-8'))
        data = client_socket.recv(1024)
        print("Ответ от сервера:", data.decode('utf-8'))
except Exception as e:
    logging.error(f"Ошибка при соединении с сервером: {e}")

```

Рисунок 11 – Взаимодействие с сервером

Результат работы клиента представлен на рисунке 12:

```

Выберите операцию:
1. Площадь трапеции
2. Площадь параллелограмма
3. Теорема Пифагора
4. Решение квадратного уравнения
Введите номер операции: 4
Введите коэффициент а: 2
Введите коэффициент b: 1
Введите коэффициент с: 1
Ответ от сервера: Нет вещественных корней

```

Рисунок 12 – Результат работы клиента

Результат работы сервера представлен на рисунке 13:

```

INFO:root:Сервер запущен и слушает 127.0.0.1:65433
INFO:root:Клиент подключен: ('127.0.0.1', 63209)
INFO:root:Получено сообщение от ('127.0.0.1', 63209): quadratic,2,1,1
INFO:root:Соединение с ('127.0.0.1', 63209) закрыто.

```

Рисунок 13 – Результат работы сервера

Задание 3

В задании реализован HTTP-сервер, который:

- Принимает GET-запрос от клиента.
- Загружает HTML-страницу из файла index.html.
- Отправляет сформированный HTTP-ответ клиенту.

Реализация сервера, который принимает соединение и отправляет html файл index.html, представлена на рисунке 14:

```
def handle_client(client_connection, client_address): 1 usage
    logging.info(f"Подключение от: {client_address}")
    try:
        request_data = client_connection.recv(1024)
        request_text = request_data.decode('utf-8')
        logging.info(f"Получен запрос:\n{request_text}")

        # Простейший анализ HTTP-запроса
        request_line = request_text.splitlines()[0]
        method, path, _ = request_line.split()
        if method != 'GET' or path != '/':
            status_line = "HTTP/1.1 405 Method Not Allowed\r\n"
            response_body = "<html><body><h1>405 Method Not Allowed</h1></body></html>"
        else:
            try:
                with open('index.html', 'r', encoding='utf-8') as file:
                    response_body = file.read()
                    status_line = "HTTP/1.1 200 OK\r\n"
            except FileNotFoundError:
                response_body = "<html><body><h1>404 Not Found</h1></body></html>"
                status_line = "HTTP/1.1 404 Not Found\r\n"

        response_headers = (
            "Content-Type: text/html; charset=utf-8\r\n"
            f"Content-Length: {len(response_body.encode('utf-8'))}\r\n"
            "\r\n"
        )
        response = status_line + response_headers + response_body
        client_connection.sendall(response.encode('utf-8'))
    except Exception as e:
        logging.error(f"Ошибка при обработке запроса от {client_address}: {e}")
    finally:
        client_connection.close()
        logging.info(f"Соединение с {client_address} закрыто.")
```

Рисунок 14 – Код server.py

Реализация клиента, который получает html файл, представлена на рисунке 15:

```
import socket
import logging

HOST = '127.0.0.1'
PORT = 65434

def main(): 1 usage
    logging.basicConfig(level=logging.INFO)
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client_socket:
            client_socket.connect((HOST, PORT))
            request = "GET / HTTP/1.1\r\nHost: {}\r\n\r\n".format(HOST)
            client_socket.sendall(request.encode('utf-8'))

            response = b""
            while True:
                chunk = client_socket.recv(1024)
                if not chunk:
                    break
                response += chunk
            print(response.decode('utf-8'))
    except Exception as e:
        logging.error(f"Ошибка при соединении с сервером: {e}")
```

Рисунок 15 – client.py

Результат работы клиента представлен на рисунке 16:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 314

<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Пример HTTP сервера</title>
</head>
<body>
  <h1>Добро пожаловать на мой сервер!</h1>
  <p>Это простая HTML-страница, загруженная с диска.</p>
</body>
</html>
```

Рисунок 16 – Результат работы клиента

Результат работы сервера представлен на рисунке 17:

```
2025-04-05 13:28:26,299 [INFO] HTTP сервер запущен и слушает 0.0.0.0:65434
2025-04-05 13:28:35,122 [INFO] Подключение от: ('127.0.0.1', 62772)
2025-04-05 13:28:35,123 [INFO] Получен запрос:
GET / HTTP/1.1
Host: 127.0.0.1

2025-04-05 13:28:35,123 [INFO] Соединение с ('127.0.0.1', 62772) закрыто.
```

Рисунок 17 – Результат работы сервера

Задание 4

В задании реализован многопользовательский чат, где:

- Клиенты подключаются к серверу.
- Каждый клиент получает уникальный ID.
- Сообщения отправляются по формату: target_id: сообщение. Если разделитель отсутствует, сообщение расценивается как broadcast.
- Сервер пересылает сообщения целевым клиентам или отправляет broadcast.

Функция для отправки широковещательного сообщения всем клиентам, кроме отправителя, представлена на рисунке 18

```
def broadcast_message(message, sender_id): 1 usage
    with lock:
        for client_id, client_socket in clients.items():
            if client_id != sender_id:
                try:
                    client_socket.sendall(f"Сообщение от {sender_id}: {message}\n".encode('utf-8'))
                except Exception as e:
                    logging.error(msg: "Ошибка отправки сообщения клиенту %s: %s", *args: client_id, e)
```

Рисунок 18 – Функция broadcast_message

Функция для обработки сообщений от конкретного клиента представлена на рисунке 19.

```
def handle_client(client_socket, client_id): 1 usage
    try:
        client_socket.sendall(f"Ваш ID: {client_id}\n".encode('utf-8'))
        while True:
            data = client_socket.recv(1024)
            if not data: ...
            message = data.decode('utf-8').strip()

            # Если сообщение не содержит ":", трактуем его как broadcast
            if ':' not in message:
                broadcast_message(message, client_id)
                client_socket.sendall("Сообщение отправлено всем.\n".encode('utf-8'))
                continue

            target_id_str, message_text = message.split(":", 1)
            target_id_str = target_id_str.strip()
            message_text = message_text.strip()
            try:
                target_id = int(target_id_str)
            except ValueError: ...

            with lock:
                if target_id in clients:
                    target_socket = clients[target_id]
                    try:
                        target_socket.sendall(f"Сообщение от {client_id}: {message_text}\n".encode('utf-8'))
                        client_socket.sendall("Сообщение отправлено.\n".encode('utf-8'))
                    except Exception as e: ...
                else: ...
        except Exception as e: ...
    finally:
        with lock:
            if client_id in clients:
                del clients[client_id]
        client_socket.close()
    logging.info(msg: "Клиент %s отключился", *args: client_id)
```

Рисунок 19 – Функция handle_client

Основной цикл для приема новых подключений представлен на рисунке 20.

```
def main(): 1 usage
    global client_id_counter
    logging.basicConfig(level=logging.INFO, format='%(asctime)s [%(levelname)s] %(message)s')
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, value=1)
    server_socket.bind((HOST, PORT))
    server_socket.listen()
    logging.info(msg: "Сервер запущен и слушает %s:%s", *args: HOST, PORT)

    try:
        while True:
            client_socket, addr = server_socket.accept()
            with lock:
                assigned_id = client_id_counter
                client_id_counter += 1
                clients[assigned_id] = client_socket
            logging.info(msg: "Новое подключение от %s. Присвоен ID: %s", *args: addr, assigned_id)
            thread = threading.Thread(target=handle_client, args=(client_socket, assigned_id), daemon=True)
            thread.start()
    except KeyboardInterrupt:
        logging.info("Сервер остановлен.")
    finally:
        server_socket.close()
```

Рисунок 20 – Основной цикл для приема новых подключений

Был написан клиент, который подключается к сокету и в бесконечном цикле отправляет сообщение. Его код представлен на рисунке 21.

```
def receive_messages(sock): 1 usage
    while True:
        try:
            data = sock.recv(1024)
            if not data:
                logging.info("Соединение с сервером потеряно.")
                break
            print(data.decode('utf-8').strip())
        except Exception as e:
            logging.error(msg: "Ошибка при получении сообщения: %s", *args: e)
            break

def main(): 1 usage
    logging.basicConfig(level=logging.INFO)
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client_socket:
            client_socket.connect((HOST, PORT))
            logging.info(msg: "Подключено к серверу %s:%s", *args: HOST, PORT)

            thread = threading.Thread(target=receive_messages, args=(client_socket,), daemon=True)
            thread.start()

            while True:
                msg = input()
                if msg.lower() == 'exit':
                    logging.info("Выход из чата.")
                    break
                client_socket.sendall(msg.encode('utf-8'))
    except Exception as e:
        logging.error(msg: "Ошибка клиента: %s", *args: e)
```

Рисунок 21 – client.py

Для проверки многопользовательского чата были запущены 2 клиента. Пример их взаимодействия представлен на рисунках 22 и 23:

```
INFO:root:Подключено к серверу 127.0.0.1:65435
Ваш ID: 2
Сообщение от 1: Привет, мир!
1:Привет, 1! Как тебя зовут?
Сообщение отправлено.
Сообщение от 1: Меня зовут Панаёт Виктор и я хочу максимальный балл за 6 лабораторную работу
1: Удачи тебе
Сообщение отправлено.
exit
INFO:root:Выход из чата.
```

Рисунок 22 – Чат первого клиента

```
INFO:root:Подключено к серверу 127.0.0.1:65435
Ваш ID: 1
Привет, мир!
Сообщение отправлено всем.
Сообщение от 2: Привет, 1! Как тебя зовут?
2: Меня зовут Панаёт Виктор и я хочу максимальный балл за 6 лабораторную работу
Сообщение отправлено.
Сообщение от 2: Удачи тебе
```

Рисунок 23 – Чат второго клиента

Логи работы сервера представлены на рисунке 24.

```
2025-04-05 14:42:01,576 [INFO] Сервер запущен и слушает 0.0.0.0:65435
2025-04-05 14:42:10,755 [INFO] Новое подключение от ('127.0.0.1', 63598). Присвоен ID: 1
2025-04-05 14:42:45,088 [INFO] Новое подключение от ('127.0.0.1', 63611). Присвоен ID: 2
2025-04-05 14:44:34,887 [ERROR] Ошибка у клиента 2: [WinError 10054] Удаленный хост принудительно разорвал существующее подключение
2025-04-05 14:44:34,887 [INFO] Клиент 2 отключился
```

Рисунок 24 – Результат работы сервера

Вывод: были освоены основы работы с сетевыми сокетами и многопоточностью в Python через разработку различных клиент-серверных приложений.